

SO SÁNH NỀN TẢNG · CÔNG TY MỘT NGƯỜI, NHIỀU AGENT

Multica hay Buzz — hay một phòng chat bạn đã có?

Hai nền tảng mã nguồn mở để điều phối một đội agent chạy song song, mở ra theo kiến trúc, mô hình vận hành và chi phí token. Kèm phương án thứ ba: dựng mô hình tương tự bằng Hermes Agent trên Telegram group.



Multica

multica-ai/multica · Go
Bảng việc cho agent

44.413

STAR

13/01/26

REPO MỞ

Go

BACKEND

v0.4.20

RELEASE

VS



Buzz

block/buzz · Rust · Apache-2.0
Phòng chat cho agent

23.650

STAR

06/03/26

REPO MỞ

Rust

BACKEND

0.5.5

DESKTOP

TRONG SỐ NÀY

- 03 Multica: bảng việc, squad, autopilot — và ranh giới dữ liệu bạn phải biết
- 05 Buzz: relay Nostr, mọi thứ là event có chữ ký — kèm 6 giới hạn repo tự thừa nhận
- 08 Bảng so sánh 14 tiêu chí, đọc được cả khi in đen trắng
- 09 Token đi đâu: bẫy đòn tiết kiệm dựa trên tài liệu, không suy diễn
- 10 Phương án thứ ba: Hermes Agent + Telegram group topic

Kết luận trước, bằng chứng sau

Hai nền tảng giải cùng một nhu cầu — một người điều phối nhiều agent chạy song song nhiều dự án — nhưng chọn hai đơn vị nguyên thuỷ khác nhau. Chọn sai đơn vị nguyên thuỷ là chọn sai nền tảng.

Multica — đơn vị nguyên thuỷ là issue		Buzz — đơn vị nguyên thuỷ là event có chữ ký	
Ảnh dụ	Bảng việc. Agent là người được gán việc.	Ảnh dụ	Phòng chat. Agent là thành viên có khoá riêng.
Vòng đời	Issue → task → runtime claim → CLI chạy → kết quả về issue	Vòng đời	Mọi thao tác là event Nostr ký số, ghi vào một log duy nhất
Song song	Nhiều task, mỗi task một run; giới hạn concurrency đặt trên agent	Song song	buzz-agent: nhiều session đồng thời một process, mặc định tối đa 8
Điều phối	Squad: một leader agent định tuyến việc cho member	Điều phối	Channel + workflow YAML; agent tự tạo phòng, gọi agent khác
Định kỳ	Autopilot: cron 5 trường hoặc webhook	Định kỳ	Workflow trigger: message · reaction · schedule · webhook
Thế mạnh	Truy vết chi phí và tiến độ theo từng run	Thế mạnh	Audit log hash-chain, danh tính bằng keypair

Ba câu hỏi quyết định, trả lời trong một phút

Việc của bạn có hình dạng “ticket”? Nếu công việc chia được thành đầu việc rời, có tiêu chí xong, cần review trước khi merge — Multica đúng hình dạng. Nếu công việc là dòng thảo luận liên tục rồi mới kết tinh thành hành động, Buzz đúng hơn.	Bạn cần audit tới mức nào? Multica cho execution log replay từng tool call kèm token cost mỗi run. Buzz cho hash-chain audit và chữ ký số trên từng event. Multica trả lời “tốn bao nhiêu”; Buzz trả lời “ai ký, không sửa được”.	Bạn chịu được bao nhiêu phần chưa xong? Buzz tự liệt kê sáu giới hạn đã kiểm chứng trong ARCHITECTURE.md, trong đó approval gate chưa nổi xong đầu-cuối. Multica ở nhịp release gần như hằng ngày nhưng giấy phép không phải OSI thuần.
--	---	---

Cả hai đều đã có mặt trong hệ Hermes, và đó là dữ kiện đáng chú ý nhất của số này: **Multica coi hermes là một trong 20 agent CLI chạy được, còn Hermes coi Buzz là một kênh messaging chính thức trong gateway.** Nghĩa là bạn không buộc phải chọn một — có đường ghép cả ba.

Nguồn: README.md multica-ai/multica (bảng Runtimes) và hermes-agent.nousresearch.com/docs/user-guide/messaging/buzz, truy xuất 06/08/2026.

Cách đọc dossier này

Trang 3–4 mô Multica, trang 5–6 mô Buzz, trang 7 đối chiếu hai triết lý, trang 8 là bảng so sánh, trang 9 nói riêng về token, trang 10 là phương án Hermes + Telegram, trang 11 là lộ trình 30 ngày. Mọi con số đều dẫn nguồn ở trang 12, kèm danh sách những điều *không* xác minh được.

HỒ SƠ KỸ THUẬT A



multica-ai/multica • Go • Multica License • 44.413 star • 5.638 fork • 1.225 issue mở

Tên là viết tắt của *Multiplexed Information and Computing Agent* — nhắc tới Multics, hệ điều hành thập niên 1960 mở ra time-sharing. Luận điểm: đội phát triển vốn đơn luồng, agent làm time-sharing quay lại, lần này người dùng chia sẻ hệ thống gồm cả người và máy.

```
Web • Desktop (macOS/Windows/Linux) • iOS
|
+-----+ +-----+ +-----+
| Next.js | --> | Go backend | --> | PostgreSQL |
| frontend | <-- | (Chi + WS) | <-- | (pgvector) |
+-----+ +-----+ +-----+
|
| tasks over WebSocket
+-----+
| Agent daemon | chạy trên máy bạn, cạnh code
+-----+
| spawns
+-----+
| Claude Code • Codex • Cursor • ... |
| (bất kỳ trong 20 runtime hỗ trợ) |
+-----+
```

Bảng khái niệm phải nắm

Workspace	Phạm vi tự chứa; người và agent cùng làm trong đó
Issue	Đơn vị việc; assignee là người, agent, hoặc squad
Project	Nhóm issue theo một mục tiêu, gắn được repo và thư mục làm context
Agent	Cấu hình tái dùng: tên, instructions, model, skill, access, runtime. Không phải process thường trú
Skill	Gói phương pháp tái dùng, gắn được cho nhiều agent, file chính SKILL.md
Runtime	Máy đã kết nối + các AI coding tool trên máy đó
Task	Một lần chạy cụ thể. Một issue sinh nhiều task theo thời gian

Vòng đời một task

deferred	Đã hẹn giờ, vào hàng đợi đúng lúc
queued	Chờ một runtime nhận
dispatched	Runtime đã nhận, đang khởi động AI coding tool
waiting_local_runtime_lock	Thư mục đích đang bị run khác giữ lock
running	Tool đang chạy
completed / failed / cancelled	Xong • lỗi hoặc bị ngắt • bị dừng tay

Hai cái bẫy vận hành

- Task treo quá 2 giờ không ai nhận sẽ thành **failed**. Runtime offline lúc bạn ngủ là mất luôn cửa sổ chạy.
- Một run **completed** không nghĩa là issue đã xong. Issue còn nhận thêm thảo luận, thêm yêu cầu, hoặc trigger lại.

Điểm kiến trúc quan trọng nhất: **Multica không ship model, cũng không ship agent CLI**. Nó điều khiển các CLI bạn đã cài và đã đăng nhập — nên đối nhà cung cấp là đối một dropdown, không phải migrate. Trong bảng 20 runtime có cả hermes, claude, codex, cursor-agent, copilot, opencode, openclaw, kimi, qwen, grok.

Nguồn: README.md multica-ai/multica, bảng Runtimes, truy xuất 06/08/2026.

Điều phối nhiều agent trong Multica

Ba cơ chế tạo nên khả năng “một người, nhiều dự án”: squad để định tuyến, autopilot để tự chạy, và ranh giới dữ liệu để bạn biết cái gì rời khỏi máy mình.

Squad — leader định tuyến, không phải chạy tất cả

Gán issue cho squad **không kích hoạt mọi agent cùng lúc**. Multica đánh thức leader trước; leader đọc context rồi quyết ai làm bước tiếp. Squad giải bài toán “việc này giao cho ai”, không tự động tăng concurrency.

1. Leader nhận task như mọi agent khác, ở lần poll kế tiếp của runtime; Multica nối ba khối brief vào system prompt lúc claim.
2. Leader chuyển issue cha sang `in_progress` và giữ nguyên trong lúc squad làm; dispatch member không tính là hoàn thành.
3. Leader đăng *một* comment delegation, @-mention member theo đúng markdown trong roster — chính mention đó sinh task mới cho từng member.
4. Leader ghi đánh giá bằng `multica squad activity <issue-id> action --reason "..."` để người thật thấy leader có thực sự cân nhắc.

Autopilot — hai chế độ, đừng chọn sai

Create issue	Mỗi trigger tạo issue rồi gán; thảo luận và lịch sử run nằm trên issue. Dùng cho việc cần người review.
Run only	Tạo task trực tiếp, không có issue; kết quả chỉ thấy trong run history. Dùng cho việc chạy nền.

Khác biệt sống còn: `create-issue` dùng chung hàng đợi — runtime offline thì issue vẫn được tạo, task chờ. `Run-only` đòi runtime sẵn sàng đúng lúc trigger, nếu không run bị skipped và không để lại gì.

Ranh giới dữ liệu

Multica giữ

- + Workspace, issue, comment, status
- + Cấu hình agent và skill
- + Trạng thái task, bản ghi run, kết quả

Máy bạn giữ

- AI coding tool và credential của chúng
- Thư mục code và file local
- Thay đổi file thật, lệnh thật

Ngoại lệ duy nhất — đọc kỹ

Mọi thứ lưu trong `custom_env` của agent **được giữ trên server Multica** và truyền cho runtime lúc thực thi. Đừng đặt vào đó secret không được phép rời máy bạn. Ranh giới này giống nhau trên cloud và self-host.

Self-host và chẩn đoán

Dựng server

Docker Compose hoặc Helm.
`install.sh --with-server` rồi `multica setup self-host`. Yêu cầu Docker, pull image từ GHCR; tag chưa publish thì fallback `make selfhost-build`.

Thành phần

Backend Go một binary (REST + WebSocket), frontend Next.js 16, PostgreSQL 17 với pgvector. Người chạy agent local cài thêm CLI `multica` và agent daemon.

Khi agent đứng

`multica version · auth status · daemon status --output json · daemon logs`. Self-host kiểm thêm `/health` và `/readyz` — chỉ `/readyz` kiểm cả DB và migration.

Multica hiển thị **token usage theo từng run, agent, issue** — với người lo tốn token, đây là thứ đáng giá nhất: thấy run nào đắt và cắt đúng chỗ.

HỒ SƠ KỸ THUẬT B



Buzz

block/buzz • Rust • Apache-2.0 • Block, Inc. • 23.650 star • 2.694 fork • 2.081 issue mở

Buzz là workspace self-host nơi người và agent chia sẻ cùng những căn phòng. Bên dưới là một relay Nostr: mọi tin nhắn, reaction, bước workflow, phê duyệt review và git event đều là một event có chữ ký trong một log duy nhất — cùng hình dạng, cùng mô hình danh tính, cùng dấu vết audit, dù tác giả là người hay tiến trình.



Bốn quyết định kiến trúc

Relay là nguồn sự thật	Mọi đọc/ghi đi qua relay. Không P2P, không gossip, không replication.
Community = URL	URL relay quyết định workspace. Self-host mặc định: một host, một relay, một community.
Thêm tính năng = thêm kind	Mỗi hành động là event Nostr có số kind. Client cũ không thấy, không vỡ.
Agent có khoá riêng	Phân quyền theo danh tính, không theo cờ permission — “cùng cách bạn scope một đồng nghiệp”.

Bản đồ crate (Rust monorepo)

Lỗi	buzz-core (kiểu zero-I/O, filter NIP-01, verify Schnorr) • buzz-relay (Axum WS + REST)
Dịch vụ	buzz-db • buzz-auth (NIP-42/98) • buzz-pubsub • buzz-search (Postgres FTS) • buzz-audit (hash-chain)
Bề mặt agent	buzz-cli (JSON in/out) • buzz-acp (harness Goose/Codex/Claude Code) • buzz-agent • buzz-dev-mcp (shell + file edit) • buzz-workflow • buzz-persona
Git	git-sign-nostr • git-credential-nostr — git ký bằng Nostr, patch theo NIP-34

Triết lý buzz-agent nói thẳng tiêu chí mà người vận hành nên soi mọi công cụ agent: **một coding agent phải nhỏ đủ để giữ trong đầu; nếu không truy được lỗi từ triệu chứng tới gốc trong vài phút thì hệ thống quá phức tạp; nếu không chạy được mười instance song song mà không lo tài nguyên thì hệ thống quá nặng**. Mặc định mỗi process gánh tối đa 8 session đồng thời, mỗi session có MCP server, history và context riêng; khi context đầy, session tự tóm tắt lịch sử của chính nó rồi chạy tiếp.

Nguồn: VISION_AGENT.md, block/buzz, truy xuất 06/08/2026.

Chạy được hôm nay — và chưa chạy được

Điểm đáng tin nhất của repo này là nó tự công bố danh sách giới hạn “đã kiểm chứng, không phải nguyện vọng thiết kế”. Dưới đây là nguyên văn, rút gọn.

Chạy được hôm nay

- + Relay, channel, thread, DM, canvas, media, search, audit log
- + Desktop app (Tauri + React)
- + `buzz-cli` agent-first JSON in/out, kèm ACP harness cho Goose · Codex · Claude Code
- + Workflow YAML: trigger message · reaction · schedule · webhook
- + Git event theo NIP-34: patch, repo announcement, status
- + Git hosting backend

Đang nối dây / chưa có

- Mobile client iOS + Android (Flutter) — đang làm
- **Approval gate chưa nối xong đầu-cuối**: engine chặn trước khi tạo bản ghi `WaitingApproval`, nên run chậm cổng phê duyệt bị đánh dấu Failed
- **Rate limiting chưa có bản thực thi**: trait tồn tại, bản duy nhất là stub luôn cho qua; bốn tier cấu hình chưa được enforce
- Action `send_dm` và `set_channel_topic` trả `NotImplemented` — run chậm vào là fail
- Huddle recording và per-track publishing: đã đặt sẵn kind, chưa có producer
- Không có sqlx offline query cache — query không được kiểm ở compile time

buzz-cli — bề mặt agent, cũng là bề mặt bạn tự động hoá

Toàn bộ output là JSON trên stdout, lỗi là JSON trên stderr. Exit code có nghĩa rõ: 0 ok · 1 lỗi người dùng · 2 network · 3 auth · 4 khác · 5 write conflict. Xác thực bằng `BUZZ_PRIVATE_KEY` theo chữ ký NIP-98.

```
buzz messages send --channel <uuid> --content "Hello"
buzz messages send --channel <uuid> --content - < message.md      # body từ stdin
buzz messages send-diff --channel <uuid> --diff - --repo <url> --commit <sha>
buzz messages search --author <pubkey|npub|name> --since <unix-ts>
buzz channels create --name "my-channel" --type stream --visibility open
buzz workflows trigger --workflow <uuid>
buzz workflows approve --token <uuid> --approved false --note "needs revision"
buzz canvas set --channel <uuid> --content "# Welcome"
```

Self-host một relay

Relay là một binary Rust duy nhất phục vụ WebSocket relay, REST API và web UI từ cùng một process. Nó cần đúng ba thứ: Postgres, Redis, và một object store tương thích S3. Có bundle Compose sản xuất trong `deploy/compose/` kèm Caddy/TLS tùy chọn; `docker-compose.yml` ở gốc chỉ dùng cho dev.

Hai khoá định danh, quan trọng hơn mọi mật khẩu

- `BUZZ_RELAY_PRIVATE_KEY` — khoá ký của chính relay. Xoay khoá là đổi pubkey được quảng bá: client đã pin khoá cũ sẽ từ chối, và mọi thứ relay từng ký không còn verify được.
- `RELAY_OWNER_PUBKEY` — khoá công khai của bạn. Buzz mặc định closed-membership; đây là tài khoản duy nhất không thể bị xoá qua `buzz-admin`.

Cùng một vấn đề, hai đơn vị nguyên thủy

Cả hai repo đều mở đầu bằng cùng một chẩn đoán: bạn đang chạy nhiều agent, mỗi cái sống trong một tab terminal, quên hết khi hết session, và bạn giải thích lại context lần thứ tư trong ngày. Chỗ hai bên rẽ nhau là câu trả lời.

Multica trả lời: đưa agent lên bảng việc

“Agent được gán issue, báo tiến độ, nêu blocker, và ship code — như đồng nghiệp người. Assignee picker, activity timeline, task lifecycle và hạ tầng runtime đều được xây quanh ý đó từ ngày đầu.”

VISION.md · multica-ai/multica

Buzz trả lời: đưa agent vào phòng

“Cảm giác như một team workspace. Bên dưới là một event log có khiêu thẩm mỹ và một số lượng crate Rust đáng ngờ. Khác biệt là những gì agent thực sự *làm được* khi đã ở bên trong.”

README.md · block/buzz

- **Ý chí là issue.** Ba câu thơ trong một issue, kết thúc là một pull request.
- **Review là cổng.** Việc đáp xuống review, không đáp vào main. Bạn quyết cái gì được ship.
- **Không review “status theatre”.** Bạn review chính thứ được làm: plan, tài liệu, diff, preview, kết quả test, và câu hỏi còn treo.
- **Lịch sử không tan theo session.** Ý định gốc, quyết định, hành động, artifact và kết quả vẫn nối với nhau.
- **Agent là thành viên, không phải bot.** Thêm agent vào channel như thêm một người.
- **Nhánh thành phòng.** Patch, CI, review và quyết định merge sống cùng một chỗ — channel trở thành bản ghi lý do code tồn tại.
- **Tìm một lần, ra tất cả.** Hội thoại, patch, workflow run và phê duyệt cùng một loại event nên cùng một index tìm kiếm.
- **Agent vận hành workspace, không chỉ nói trong đó.** Tạo channel, sửa canvas, chạy workflow, vào huddle.

Ba tình huống, đặt cạnh nhau

TÌNH HUỐNG	MULTICA XỬ LÝ	BUZZ XỬ LÝ
Ba dự án song song	Ba project, mỗi project gắn repo và thư mục riêng; issue chảy trong project. Concurrency giới hạn trên từng agent.	Ba channel (hoặc ba community). Agent là member nhiều phòng, mỗi session giữ context riêng, tối đa 8 session/process.
“Việc này đã gặp chưa?”	Tìm trong issue và execution log; context bám vào issue nên đọc lại được nguyên nhân.	Agent quét lịch sử channel rồi đăng chính các thread làm bằng chứng — hỏi trong phòng, trả lời trong phòng.
Việc định kỳ 6h sáng	Autopilot: cron 5 trường + IANA timezone, hoặc webhook. Chọn create-issue nếu cần review.	Workflow YAML trigger schedule. Lưu ý một số action còn <code>NotImplemented</code> .
Chọn agent làm bộ	Review gate + access scope theo role <code>owner / admin / member</code> , và chọn agent nào member được chạy.	Scope theo danh tính keypair và membership channel. Nhưng approval gate <i>chưa</i> nối xong đầu-cuối.

Nhận định thẳng: **Multica** chín hơn ở phần “quản trị công việc”, **Buzz** chín hơn ở phần “nền tảng giao tiếp”. Nếu bạn đang mất thời gian vì không biết agent nào làm gì tới đâu, vấn đề của bạn là quản trị việc. Nếu bạn mất thời gian vì thông tin nằm rải rác bảy tab, vấn đề của bạn là nền tảng giao tiếp.

Đối chiếu trực diện

■ Mạnh / đã chạy ■ Có nhưng có điều kiện ■ Thiếu / chưa xong

Bảng có nhân chữ ở mỗi ô nên in đen trắng vẫn đọc được nghĩa.

	 Multica Go · bảng việc	 Buzz Rust · relay Mostr	 Hermes + Telegram Python · gateway chat
Đơn vị nguyên thủy	Issue → task → run	Event Nostr có chữ ký	Session theo chat/topic
Giấy phép	Multica License: Apache-2.0 + điều kiện về dịch vụ hosted, nhúng thương mại, thương hiệu	Apache-2.0 thuần	MIT
Ngôn ngữ · hạ tầng	Go + Next.js 16 + PostgreSQL 17/pgvector	Rust, một binary + Postgres + Redis + S3/MinIO	Python; VPS nhỏ là đủ
Star · fork (06/08/26)	44.413 · 5.638	23.650 · 2.694	226.373 · —
Issue mở	1.225	2.081	—
Chạy song song	Nhiều task; concurrency limit đặt trên agent; có directory lock	Tối đa 8 session/process, MCP riêng từng session	delegate_task mặc định 3 subagent đồng thời
Điều phối nhiều agent	Squad có leader định tuyến bằng @-mention	Agent gọi agent qua channel; điều phối do bạn viết	Subagent leaf không tự delegate; nesting phải bật
Việc định kỳ	Autopilot: cron 5 trường + IANA tz, webhook, 2 chế độ	Workflow YAML schedule; vài action còn NotImplemented	cronjob: cron/interval, gán skill, chọn nơi giao kết quả
Cổng phê duyệt	Review gate: việc đáp xuống review, không vào main	Chưa nối xong đầu-cuối — run chậm cổng bị Failed	Approval prompt cho lệnh nguy hiểm; YOLO mode tắt được
Audit	Execution log replay từng tool call, có timestamp	Hash-chain audit + chữ ký Schnorr từng event	SQLite state.db + FTS, session_search
Theo dõi token/chi phí	Token usage theo run, theo agent, theo issue	Không thấy tài liệu về đo chi phí per-run	Status bar token/context/cost; /usage , /context
Rate limit	—	Trait có, bản duy nhất là stub luôn cho qua	—
Vào từ đâu	Web · Desktop (Electron) · iOS build từ source · Slack/Lark/DingTalk	Desktop Tauri · CLI · relay URL; mobile đang làm	21+ nền tảng chat, gồm cả Buzz
Khoá nhà cung cấp	Không ship model; điều khiển 20 agent CLI đã cài	Đổi LLM provider bằng một biến môi trường	300+ model qua Portal, hoặc endpoint tự host

Chốt lại

Chọn Multica nếu bạn cần biết chính xác từng run tốn bao nhiêu và ai duyệt cái gì — nó là hệ thống quản trị công việc có agent bên trong. **Chọn Buzz** nếu bạn cần một nền tảng nơi người và agent bình đẳng về danh tính, mọi thứ ký số và tìm được trong một log — và bạn chịu được phần chưa xong. **Chọn Hermes + Telegram** nếu ưu tiên số một là chi phí vận hành thấp và bạn muốn điều khiển mọi thứ từ điện thoại.

Token đi đâu, và cắt ở đâu

Cả hai nền tảng đều tốn token vì cùng một lý do: chúng làm agent *chủ động hơn*, và mỗi lần chủ động là một vòng nạp lại context. Dưới đây là sáu đòn có cơ sở tài liệu, không phải mẹo suy diễn.

Ba nguồn tốn token lớn nhất

1 • Nạp lại context mỗi run

Trong Multica, mỗi trigger sinh một task mới và runtime khởi động lại AI coding tool. Instructions của agent được dùng ở **mọi run**. Prompt dài × số run = hoá đơn.

2 • Output tool đổ vào context

Log dài, file to, kết quả search — đều thành token đầu vào ở lượt sau. Hermes chặn bằng ba mức: `max_bytes 50000`, `max_lines 2000`, `max_line_length 2000`.

3 • Schema tool luôn nằm trong prompt

Nhiều MCP server = nhiều JSON schema gửi mỗi lượt, dù không dùng. Hermes có Tool Search để hoãn schema; hai nền tảng kia không tài liệu hoá cơ chế tương đương.

Sáu đòn tiết kiệm, theo thứ tự hiệu quả

ĐÒN 1 • MỌI NỀN TẢNG

Đưa hướng dẫn lặp lại ra file, không nhét vào prompt

Multica tách rõ: *instructions* thuộc một agent và gửi ở mọi run, còn *skill* gắn được cho nhiều agent và bật/tắt từng cái. Hermes dùng `AGENTS.md` và skill nạp theo yêu cầu. Nguyên tắc: chỉ để trong instructions những gì đúng ở *mọi run*.

ĐÒN 2 • HERMES

Bật Tool Search khi có nhiều MCP

Tool MCP và plugin ngoài lõi bị thay bằng ba tool bridge: `tool_search`, `tool_describe`, `tool_call`. Model nạp schema theo nhu cầu thay vì gán cả rổ mỗi lượt. Tool lõi không bao giờ bị hoãn.

ĐÒN 3 • HERMES

Hạ ngưỡng nén, giữ đuôi ngắn

Mặc định nén ở `threshold 0.50` context, giữ `target_ratio 0.20` làm đuôi và `protect_last_n 20` tin nhắn; có `idle_compact_after_seconds` để nén khi rảnh. Session chat dài chạy ngầm là nơi nén giúp nhiều nhất.

ĐÒN 4 • MULTICA

Đọc token usage rồi cắt đúng agent, và dùng run-only

Chi phí xem được theo từng run, agent, issue — dữ liệu để hạ model cho agent tổng hợp tin và giữ model mạnh cho agent viết code, thay vì hạ đồng loạt. Song song đó, việc chạy nền không ai đọc thì để autopilot ở run-only: create-issue kéo theo thảo luận, và mỗi comment là context của lần chạy sau.

ĐÒN 5 • BUZZ

Để session tự tóm tắt, và giới hạn số session

`buzz-agent` tự tóm tắt history khi context đầy rồi chạy tiếp, và cap session đồng thời (mặc định 8) cấu hình được. Hạ cap là hạ đồng thời cả trần token đang cháy song song.

ĐÒN 6 • HERMES & NGUYÊN TẮC CHUNG

Đừng tiêu lượt model cho việc shell — và giao việc có tiêu chí xong

Trong CLI, tiền tố `!` chạy lệnh shell mà **không gọi model**: không API call, không token, không vào history, không phá prompt cache. Còn đòn rẻ nhất không cần cấu hình gì: một issue mô tả rõ đầu ra chạy một vòng, một issue mơ hồ chạy ba vòng rồi vẫn phải sửa tay — ba vòng đó là ba lần nạp lại toàn bộ context.

Điều chúng tôi không xác minh được

Không có tài liệu công bố nào của Multica hay Buzz đưa ra **con số cụ thể** về token tiêu thụ trung bình mỗi run, cũng không có trang giá công khai của Multica (`multica.ai/pricing` trả 404 ngày 06/08/2026). Dossier này vì vậy không đưa ra ước tính USD/tháng nào.

PHƯƠNG ÁN THỨ BA



Dựng mô hình tương tự bằng Hermes + Telegram

NousResearch/hermes-agent · Python · MIT · 226.373 star

Ý tưởng bạn nêu là khả thi, và điểm mấu chốt nằm ở một tính năng cụ thể: **session của Hermes tách theo chat và theo topic**. Mỗi Telegram topic là một không gian làm việc riêng, giữ context riêng — đúng vai trò “project” của Multica hay “channel” của Buzz.

Bốn mảnh ghép có sẵn

Topic = dự án	Telegram Bot API 9.4 (02/2026) cho phép topic dạng forum ngay trong DM 1-1 , không cần supergroup. Cấu hình ở <code>platforms.telegram.extra.dm_topics</code> .
Group = phòng họp	Trong group: bật <code>require_mention</code> để bot chỉ trả lời khi được gọi, kèm <code>observe_unmentioned_group_messages</code> để nó vẫn <i>đọc</i> ngữ cảnh mà không tiêu lượt chạy.
Cron = autopilot	<code>cronjob</code> gắn được skill, chọn nơi giao kết quả. Ở chế độ topic phải đặt <code>TELEGRAM_CRON_THREAD_ID</code> , nếu không cron rơi vào lobby hệ thống và trả lời ở đó không mở session.
Subagent = squad	<code>delegate_task</code> sinh agent con context riêng, terminal riêng, toolset hạn chế; mặc định 3 con song song, cấu hình được.

Bố cục topic để xuất

TOPIC	VAI TRÒ
<code>#cron</code>	Nơi cron giao kết quả; trả lời trong đây là tiếp session của topic
<code>#dự-án-A</code>	Một workdir, một context; cron riêng cho dự án này
<code>#dự-án-B</code>	Tách hẳn khỏi A để context không lẫn
<code>#nghiên-cứu</code>	Chỉ web + file toolset, model rẻ
<code>#duyệt</code>	Nơi bạn xem output trước khi cho chạm dữ liệu thật

Ba cái bẫy đã có tài liệu

- **Topics phải bật từ client Telegram** — bot không tự bật được. Chưa bật thì log ghi “The chat is not a forum” và bỏ qua việc tạo topic.
- **Privacy mode của bot bật mặc định**: trong group nó chỉ thấy lệnh `/`, reply trực tiếp và service message. Đây là nguồn nhầm lẫn phổ biến nhất.
- **Đổi `require_mention` phải restart gateway từ shell khác** — tiến trình gateway không tự restart chính nó.

Nên chọn kiến trúc nào — theo ba trục

MULTICA DẪN KHI

- > Bạn cần con số chi phí theo từng run
- > Có nhiều repo và cần review gate thật
- > Bạn muốn đổi agent CLI như đổi dropdown
- > Có người thứ hai cùng làm, cần role/scope

BUZZ DẪN KHI

- > Bạn cần audit không thể sửa và chữ ký số
- > Muốn agent tự vận hành phòng, không chỉ nói
- > Chấp nhận tự viết phần điều phối
- > Thích một binary Rust dễ đọc, dễ fork

HERMES + TELEGRAM DẪN KHI

- > Ưu tiên chi phí vận hành thấp nhất
- > Bạn điều khiển từ điện thoại, mọi lúc
- > Cần cron + memory + skill ngay, không dựng hạ tầng
- > Muốn ghép dẫn: Hermes làm agent *trong* Multica, hoặc nói chuyện *qua* Buzz

Đường ghép đáng thử nhất, vì cả hai đầu đều đã có tài liệu: **giữ Hermes làm agent cá nhân trên Telegram**, đồng thời **khai báo hermes làm runtime trong Multica** cho việc có repo và cần review gate. Cần audit ký số thì bật thêm adapter Buzz trong gateway. Ba lớp, một agent, không phải xây lại.

Bốn tuần để chọn xong, bằng dữ liệu của bạn

Bạn đã dùng cả hai, nên lộ trình này không phải để học lại — nó để *đo*, và để chốt bằng số thay vì bằng cảm giác. Mỗi tuần có một mục tiêu kiểm chứng được.

TUẦN 1

Đo trước khi tối ưu

1. Chọn đúng ba luồng việc thật đang chạy tay mỗi tuần, mỗi luồng 2–5 giờ.
2. Trên Multica: chạy cả ba bằng agent, ghi lại token usage theo run và theo agent.
3. Trên Hermes: chạy cùng ba luồng trong ba topic riêng, xem `/usage` và `/context`.
4. Không tối ưu gì trong tuần này. Chỉ ghi số.

Xong khi: có bảng ba luồng × hai nền tảng, kèm token và thời gian thật.

TUẦN 2

Cắt token bằng bốn đòn rẻ nhất

1. Rút gọn instructions, đẩy phần lặp lại vào skill / `AGENTS.md`.
2. Bật Tool Search nếu đang gắn nhiều MCP.
3. Chuyển autopilot không cần bàn sang chế độ run-only.
4. Dùng `!` shell mode cho mọi việc kiểm tra trạng thái.

Xong khi: đo lại ba luồng và so được % token giảm so với tuần 1.

TUẦN 3

Thử điều phối nhiều agent thật

1. Multica: lập một squad ba member, gán một issue mơ hồ có chủ đích, xem leader định tuyển ra sao.
2. Hermes: cùng việc đó, chạy bằng `delegate_task` ba subagent song song.
3. Buzz: nếu relay đã dựng, thử biến một nhánh thành phòng và cho agent review vòng đầu.
4. Ghi lại: bao nhiêu lần bạn phải can thiệp tay.

Xong khi: biết mô hình nào cần bạn can thiệp ít nhất trên cùng một việc.

TUẦN 4

Chốt kiến trúc và khoá lại

1. Chọn nền tảng chính theo số của tuần 1–3, không theo cảm tính.
2. Dựng self-host cho nền tảng đã chọn; kiểm `/readyz` hoặc health check tương ứng.
3. Rà quyền: cái gì agent được ghi, cái gì phải qua bạn duyệt.
4. Viết lại ba luồng thành skill có tiêu chí xong rõ ràng, để lần sau chạy một vòng là xong.

Xong khi: ba luồng chạy tự động, có nơi duyệt, và bạn biết mỗi luồng tốn bao nhiêu.

Năm cái bẫy nên tránh ngay từ tuần 1

1. Đặt secret vào `custom_env` của agent Multica — nó nằm trên server, không nằm trên máy bạn.
2. Dựa vào approval gate của workflow Buzz khi nó chưa nối xong đầu-cuối, hoặc trông vào rate limit của Buzz để chặn agent (bản hiện tại luôn cho qua).
3. Để runtime Multica offline qua đêm rồi ngạc nhiên vì task treo quá 2 giờ thành `failed`; hoặc coi run `completed` là issue đã xong.
4. Dùng autopilot run-only cho việc quan trọng: runtime offline lúc trigger là run bị skipped, không để lại gì.
5. Bật Telegram group cho bot mà quên privacy mode và `require_mention` — hoặc bot không thấy gì, hoặc nó trả lời mọi thứ và đốt token.

Một lời cuối về chi phí: phần lớn hoá đơn không đến từ nền tảng — nó đến từ **việc giao không rõ tiêu chí xong**, buộc agent chạy nhiều vòng và nạp lại toàn bộ context mỗi vòng. Đòn tiết kiệm mạnh nhất vẫn là viết để bài tốt hơn, và nó miễn phí.

Nguồn tham khảo

Toàn bộ số liệu trong dossier được lấy trực tiếp từ GitHub API và tài liệu chính thức của ba dự án, truy xuất ngày 06/08/2026. Không có con số nào là ước tính của người biên soạn.

Multica

Metadata repo, star, fork, issue, release api.github.com/repos/multica-ai/multica

README: 20 runtime, kiến trúc, self-host raw.githubusercontent.com/multica-ai/multica/main/README.md

VISION.md: luận điểm time-sharing [.../main/VISION.md](https://main/VISION.md)

SELF_HOSTING.md: thành phần, cài đặt [.../main/SELF_HOSTING.md](https://main/SELF_HOSTING.md)

Docs: concepts · agents · tasks · squads · autopilots · skills · how-multica-works · cli [multica.ai/docs/...](https://multica.ai/docs/)

Buzz

Metadata repo, license Apache-2.0, release api.github.com/repos/block/buzz

README: bảng “works today / being wired up”, quick start raw.githubusercontent.com/block/buzz/main/README.md

ARCHITECTURE.md: executive summary, crate reference, mục 9 Known Limitations [.../main/ARCHITECTURE.md](https://main/ARCHITECTURE.md)

VISION_AGENT.md: buzz-agent, cap 8 session [.../main/VISION_AGENT.md](https://main/VISION_AGENT.md)

buzz-cli README: lệnh, exit code, auth [.../main/crates/buzz-cli/README.md](https://main/crates/buzz-cli/README.md)

Block Engineering: “Run your own Buzz relay”, 31/07/2026 engineering.block.xyz/blog/run-your-own-buzz-relay

Hermes Agent

Metadata repo, MIT, star api.github.com/repos/NousResearch/hermes-agent

Features overview: cron, subagent, memory, skill hermes-agent.nousresearch.com/docs/user-guide/features/overview

Tools & Toolsets; Tool Search [.../features/tools](https://features/tools) · [.../features/tool-search](https://features/tool-search)

Configuration: compression, tool_output [.../user-guide/configuration](https://user-guide/configuration)

Messaging gateway: bảng nền tảng, có dòng Buzz [.../user-guide/messaging/](https://user-guide/messaging/)

Telegram: topics, privacy mode, require_mention, cron thread [.../user-guide/messaging/telegram](https://user-guide/messaging/telegram)

Buzz adapter: relay_url, BUZZ_PRIVATE_KEY, transport [.../user-guide/messaging/buzz](https://user-guide/messaging/buzz)

CLI interface: status bar token/cost, ! shell mode [.../user-guide/cli](https://user-guide/cli)

Những điều không xác minh được

- Giá Multica.** multica.ai/pricing trả HTTP 404 ngày 06/08/2026. Trang chủ chỉ có “Start free trial” và “Talk to sales”. Không đưa ra con số giá nào.
- Mức token trung bình mỗi run.** Không có tài liệu công bố cho cả Multica và Buzz. Sáu đòn ở trang 9 dựa trên cơ chế có tài liệu, không dựa trên benchmark.
- Yêu cầu tài nguyên tối thiểu.** Không tìm thấy con số RAM/CPU khuyến nghị cho self-host của cả hai.
- Logo Buzz.** Dùng app icon từ desktop/src-tauri/icons/ trong repo vì không có file logo chính thức trong docs/assets. Đã kiểm bằng vision: đúng hình con ong vàng trên nền đen.
- Star Hermes 226.373** lấy từ GitHub API cùng ngày, repo NousResearch/hermes-agent.

Phương pháp

Số liệu repo lấy bằng GitHub REST API v3, không qua trang web, để tránh sai lệch do cache. Các khẳng định về giới hạn của Buzz dịch trực tiếp từ mục “Known Limitations — verified gaps in the current implementation, not design aspirations”.